

Materia: Programación Funcional	Código: 72.60
Créditos: 3	Carga horaria total: 51
Departamento: Sistemas Digitales y Datos	Año: 2023
Carrera de: Ingeniería en Informática	
Fecha última actualización: 21/12/2022 12:48:01	

➤ Carga horaria:

51	Horas totales
34	hs. teóricas
17	hs. prácticas
0	hs. de laboratorio

3	Horas semanales
3	hs. presencial
0	hs. a distancia

➤ Contenidos mínimos:

Características del Paradigma Funcional. Transparencia referencial. Mecanismo para construir tipos de datos recursivos y polimórficos. Estructuras de datos infinitas para funciones de orden superior. Cálculo lambda. Reducciones alpha, beta, eta, delta. Punto Fijo. Formas de evaluación. Introducción a la semántica computacional. Tipos de semántica: denotacional, axiomática, operacional. Método formal para especificación y verificación de programas. Taller de aplicación con un lenguaje funcional tipo Lisp, Hope, Haskell.

➤ Presentación de la materia:

La materia de Programación Funcional busca presentar conceptos básicos del paradigma Funcional con un abordaje que sea preciso pero que no implique una formalización teórica, con el objetivo de ampliar la comprensión del fenómeno de la computación y mejorar en habilidades de programación a través del manejo de estos conceptos.

Es una materia importante, pues la Programación Funcional tiene un enfoque diferente al visto en otros paradigmas, por lo que es un complemento importante para comprender mejor las prácticas de programación. Además, en los últimos tiempos las técnicas de programación funcional se han ido incorporando parcialmente en lenguajes de uso cotidiano, por lo que se vuelve aún más importante conocer y manejar alguno de estos conceptos.

Se trabaja en clases semanales presenciales de 3 horas, con un principio de aula invertida gracias al apoyo de clases teóricas grabadas, y con diversos mecanismos disponibles en el campus, tales como cuestionarios y guías de trabajos prácticos para reforzar los aspectos trabajados.

Es una materia optativa de los últimos años de la carrera de Ingeniería en Sistemas. Aprovecha el manejo previo de programación y estructuras de datos, pero puede funcionar en forma autocontenida.

Los contenidos se organizan en 4 partes: una primera parte de presentación de herramientas básicas y familiarización

con las mismas, una segunda parte para trabajar el importante concepto de inducción y recursión estructural, una tercera parte para incluir el abordaje de programas puros que expresen interacción con el medio, y finalmente una última parte con consideraciones teóricas sobre la importancia de las funciones de orden superior. Se utiliza como herramienta el lenguaje de programación Haskell, pero los conceptos presentados trascienden dicho lenguaje.

➤ Objetivos de aprendizaje:

- Mejorar la comprensión de la computación y en particular de la programación a través de conocer características de un paradigma diferente a los tradicionales.
- Confeccionar programas sencillos utilizando nociones fundamentales del paradigma funcional.
- Demostrar propiedades sencillas de programas funcionales utilizando inducción estructural.

➤ Contenidos:

Título	Descripción
Presentación del Paradigma Funcional.	Modelo de cómputo funcional. Visiones denotacional y operacional. Funciones de orden superior. Modelo de computación puro (sin efectos laterales). Relevancia de estas características.
Sistemas de tipos	Definición de la noción de sistemas de tipos, su propósito y sus características. Sistemas de tipos Hindley-Milner simple y polimórfico.
Currificación	Aplicación del orden superior para mejorar la expresividad de las funciones. Formas de definición y notación de funciones currificadas. Ventajas y desventajas de su utilización.
Reducción	Mecanismo de cómputo funcional: definición precisa y estudio de sus características. Existencia de no terminación y errores de ejecución, y su comprensión dentro de las visiones denotacional y operacional. Bottom. Funciones parciales y totales. Funciones estrictas y no estrictas y su vinculación a las formas de reducción.
Tipos algebraicos	Extensión del sistema de tipos Hindley-Milner con la definición de tipos algebraicos. Constructores y pattern matching. Características de los tipos algebraicos, y clasificación de diferentes tipos algebraicos.
Propiedades y demostraciones	Formas de demostrar propiedades sobre funciones y expresiones. Principio de Extensionalidad. Importancia de la posibilidad de demostrar propiedades de forma ecuacional.

Inducción y recursión estructural	Principio de Inducción estructural como mecanismo para definir tipos algebraicos con infinitos elementos, funciones recursivas sobre dichos tipos y demostrar propiedades sobre los mismos. Importancia de la noción de estructura .
Tipos de datos recursivos.	Tratamiento de tipos recursivos clásicos: listas y árboles. Definición de funciones por recursión estructural sobre listas y árboles.
Ejemplos de aplicación	Análisis de la computación como manipulación simbólica de expresiones con significado. Definición de un intérprete de un lenguaje simple de programación imperativa a través de dichas ideas.
Esquemas de funciones	Aplicación de las funciones de orden superior para la expresión de esquemas de funciones que permiten expresar infinitas soluciones diferentes como casos particulares del mismo esquema. Importancia teórica de los esquemas y aplicaciones prácticas.
Mónadas	Formas de expresar efectos en un modelo de programación puro sin perder la pureza. Definición de mónadas. Uso de mónadas para expresar entrada/salida en programas funcionales.
Lambda cálculo	Definición de un lenguaje minimalista para expresar cómputo. Muestra del poder de dicho lenguaje, expresando construcciones tradicionales en el mismo.

➤ Estrategias de enseñanza:

Se trabaja con un esquema de aula invertida en clases presenciales semanales de 3hs, a través de videos pregrabados de material teórico. Se ejercitan estos conocimientos a través de la propuesta de guías de ejercicios prácticos, y se refuerzan los conocimientos adquiridos a través de cuestionarios semanales automatizados.

➤ Actividades:

Título	Descripción
Videos con contenido teórico	Videos de dictados virtuales anteriores para presentar los contenidos teóricos de la materia.
Clases presenciales	Se trabaja con mecanismo de aula invertida, sobre lo aprendido con los videos de contenido teórico, reforzando los conceptos y resolviendo dudas.
Guías de trabajos prácticos	Se proponen ejercicios prácticos en forma semanal para poner en práctica los conceptos trabajados cada semana.
Cuestionarios de autoevaluación	Cuestionarios automatizados de teoría y práctica para la autoverificación en la adquisición de los conceptos clave, y para permitir un seguimiento de avances.
Consultas asincrónicas a través de un canal de Discord	Comunicación permanente en forma asincrónica para la resolución de dudas.

➤ Recursos didácticos:

- Guías de ejercicios
- Cuestionarios
- Canal de Discord

➤ Modalidad de evaluación y aprobación:

Modalidad de evaluación:

Se evalúa en forma permanente por la participación en clase a través de preguntas y de ejercicios prácticos propuestos.

Luego de la semana 12 se propone una evaluación parcial consistente en ejercicios prácticos que sintetizan los conceptos trabajados (que puede ser recuperada en una fecha posterior), con lo que se regulariza la cursada. La materia se completa con la confección de un trabajo de desarrollo de índole práctica a modo de final, incluyendo los contenidos finales no cubiertos en el parcial. La naturaleza exacta del trabajo depende de cada estudiante y se decide en conjunto con la cátedra. Una vez realizado el trabajo se debe defender en forma oral en una de las fechas válidas de final.

Requisitos de aprobación:

Participación regular en las clases presenciales.

Aprobación del parcial (o su recuperatorio) con nota 6 o más.

Aprobación del trabajo práctico final.

➤ Bibliografía obligatoria:

N°	Descripción
1	Bird, Richard; Wadler, Philip. Introduction to functional programming. Prentice Hall, Oxford, 1988
2	Bird, Richard. Introduction to functional programming using Haskell, Second Edition. Prentice Hall, Oxford, 1998
3	Davie, A.. An Introduction to functional programming systems using Haskell. Cambridge University Press, 1992
4	Hudak, Perterson y Fasel. A gentle introduction to Haskell, First Edition. Prentice Hall, 2000
5	Hughes, John. Why Functional Programming matters?. Computer Journal 32 (2), 1989
6	Lipovaca, Miran. Learn you a Haskell for great good. No Starch Press Inc., 2011. URL: http://learnyouahaskell.com/

➤ Bibliografía complementaria:

N°	Descripción
1	Peter A. Fejer y Dan A. Simovici. Mathematical Foundations of Computer Science Volume I: Sets, Relations, and Induction. Springer Verlag, 1991

2	Bird, Richard y Gibbons, Jeremy. Algorithm design with Haskell. Cambridge University Press, 2020.
3	Bragilevsky, Vitaly. Haskell in depth. Manning, 2021.
4	Bird, Richard. Pearls of functional programming design. Cambridge University Press, 2010.